Programmation Shell et automatisation

Systèmes d'exploitation — L1 Mathématiques / Informatique

Axel Guérin

Objectifs du cours

À l'issue de ce cours, vous devrez comprendre :

- le rôle d'un Shell dans un système d'exploitation ;
- la différence entre terminal, Shell et script ;
- le fonctionnement de l'interprétation d'un script ;
- les variables et les arguments ;
- les conditions et les tests ;
- les boucles et l'automatisation ;
- le lancement de programmes depuis un script ;
- les codes de retour et la gestion des erreurs.

Idée centrale

Le Shell est à la fois une interface permettant de communiquer avec le système et un langage permettant d'automatiser des tâches.

Plan du cours

1. Le Shell et son fonctionnement
   - Terminal, Shell et commandes
   - Les principaux Shells
   - Les commandes et leurs arguments
2. Les scripts Shell
   - Définition et intérêt
   - Interprétation d'un script
   - Le shebang et les permissions
3. Variables et arguments
   - Les variables
   - Les guillemets
   - Arguments positionnels
   - Variables d'environnement
4. Conditions et tests
   - Principe des conditions
   - Tests numériques et textuels

Chapitre 1 — Le Shell et son fonctionnement

1. Le Shell et son fonctionnement
   - Terminal, Shell et commandes
   - Les principaux Shells
   - Les commandes et leurs arguments
2. Les scripts Shell
   - Définition et intérêt
   - Interprétation d'un script
   - Le shebang et les permissions
3. Variables et arguments
   - Les variables
   - Les guillemets
   - Arguments positionnels
   - Variables d'environnement
4. Conditions et tests
   - Principe des conditions
   - Tests numériques et textuels

Terminal, Shell et commandes

Trois notions doivent être distinguées :

Terminal Application permettant d'interagir avec une interface textuelle.

Shell Programme qui interprète les commandes saisies par l'utilisateur.

Commande Instruction demandant l'exécution d'une opération.

$$\text{Utilisateur} \longrightarrow \text{Terminal} \longrightarrow \text{Shell} \longrightarrow \text{Système}$$

Le Shell : une interface avec le système

Le Shell permet notamment :

- de lancer des programmes ;
- de manipuler les fichiers ;
- de naviguer dans les répertoires ;
- de gérer certains processus ;
- de transmettre des arguments ;
- d'automatiser des tâches.

Le Shell n'est pas le noyau du système : il utilise les services du système d'exploitation pour effectuer les opérations demandées.

Les principaux Shells

Il existe plusieurs interpréteurs de commandes.

Shell	**Caractéristiques**
`sh`	Shell historique et interface POSIX
`bash`	Très utilisé sous GNU/Linux
`zsh`	Shell moderne, courant sous macOS
`fish`	Orienté interaction et ergonomie
PowerShell	Shell et langage d'automatisation Microsoft

Dans ce cours, nous utiliserons principalement **Bash**, disponible sous Ubuntu.

Structure d'une commande

Une commande suit généralement la structure :

`commande options arguments`

Exemple :

```
ls -l /home
```

- `ls` : programme ou commande ;
- `-l` : option ;
- `/home` : argument indiquant un répertoire.

Les arguments permettent de préciser le comportement d'une commande.

Commandes internes et externes

Toutes les commandes ne fonctionnent pas de la même manière.

Commandes internes

Elles sont directement prises en charge par le Shell.
Exemples : `cd`, `echo`, `read`.

Commandes externes

Elles correspondent à des programmes que le Shell peut lancer.
Exemples : `ls`, `python3`, `mkdir`.

La commande `type` permet d'obtenir des informations sur la nature d'une commande.

Chapitre 2 — Les scripts Shell

1. Le Shell et son fonctionnement
   - Terminal, Shell et commandes
   - Les principaux Shells
   - Les commandes et leurs arguments
2. Les scripts Shell
   - Définition et intérêt
   - Interprétation d'un script
   - Le shebang et les permissions
3. Variables et arguments
   - Les variables
   - Les guillemets
   - Arguments positionnels
   - Variables d'environnement
4. Conditions et tests
   - Principe des conditions
   - Tests numériques et textuels

Qu'est-ce qu'un script Shell ?

Un script Shell est un fichier texte contenant des instructions destinées à être interprétées par un Shell.

Il permet notamment :

- d'automatiser des opérations répétitives ;
- d'enchaîner plusieurs commandes ;
- de préparer un environnement ;
- de lancer des programmes ;
- de traiter des ensembles de fichiers.

Un script permet de transformer une succession d'actions manuelles en une procédure reproductible.

Un premier script

Considérons le fichier `bonjour.sh` :

```
#!/bin/bash

echo "Bonjour"
pwd
date
```

Le Shell lit les instructions et les exécute dans l'ordre.

Pour lancer ce script :

```
bash bonjour.sh
```

Que se passe-t-il pendant l'exécution ?

Lorsqu'un script est exécuté :

1. le Shell lit les instructions ;
2. il interprète leur syntaxe ;
3. il effectue les substitutions nécessaires ;
4. il exécute les commandes ;
5. il récupère leurs résultats et codes de retour.

Fichier $\longrightarrow$ Interpréteur Bash $\longrightarrow$ Exécution

Le shebang

La première ligne d'un script peut être :

```
#!/bin/bash
```

Cette ligne est appelée **shebang**.
Elle indique au système l'interpréteur à utiliser lors de l'exécution directe du fichier.

Autre écriture courante :

```
#!/usr/bin/env bash
```

Cette seconde forme recherche `bash` dans l'environnement d'exécution.

Exécution et permissions

Deux méthodes principales :

1. Lancer explicitement Bash :

```
bash script.sh
```

2. Exécuter directement le fichier :

```
chmod +x script.sh
./script.sh
```

Différence

La première méthode lance Bash avec le fichier en argument.
La seconde demande au système d'exécuter directement le fichier.

Chapitre 3 — Variables et arguments

1. Le Shell et son fonctionnement
   - Terminal, Shell et commandes
   - Les principaux Shells
   - Les commandes et leurs arguments
2. Les scripts Shell
   - Définition et intérêt
   - Interprétation d'un script
   - Le shebang et les permissions
3. Variables et arguments
   - Les variables
   - Les guillemets
   - Arguments positionnels
   - Variables d'environnement
4. Conditions et tests
   - Principe des conditions
   - Tests numériques et textuels

Les variables Shell

Une variable permet d'associer un nom à une valeur.

```
nom="Alice"
age=18
```

Pour accéder à sa valeur :

```
echo "$nom"
echo "$age"
```

Syntaxe

En Bash, l'affectation ne comporte pas d'espaces autour du signe =.

Substitution des variables

Le Shell remplace les références aux variables par leur valeur.

```
nom="Alice"

echo "Bonjour $nom"
```

Résultat :

```
Bonjour Alice
```

La notation ${nom} permet de délimiter explicitement le nom :

```
echo "${nom}_resultat.txt"
```

Guillemets simples et doubles

En Bash, les guillemets ont des comportements différents.

```
nom="Alice"

echo "Bonjour $nom"
echo 'Bonjour $nom'
```

Résultat :

```
Bonjour Alice
Bonjour $nom
```

À retenir

Les guillemets doubles autorisent la substitution des variables.

Pourquoi protéger les variables ?

Considérons :

```
fichier="mes donnees.txt"
```

Une bonne pratique consiste à écrire :

```
python3 programme.py "$fichier"
```

Les guillemets empêchent ici le Shell de découper la valeur en plusieurs arguments.

Bonne pratique

Utilisez généralement des guillemets doubles autour des expansions de variables.

Arguments d'un script

Un script peut recevoir des arguments au moment de son lancement.

```
./bonjour.sh Alice 18
```

Dans le script :

```
echo "$1"
echo "$2"
echo "$#"
```

Variable	Signification
$0	nom du script
$1	premier argument
$2	deuxième argument
$#	nombre d'arguments

Les variables d'environnement

Certaines variables décrivent l'environnement d'exécution.

Exemples :

```
echo "$HOME"
echo "$USER"
echo "$PATH"
echo "$SHELL"
```

- `HOME` : répertoire personnel ;
- `USER` : utilisateur ;
- `PATH` : répertoires de recherche des programmes ;
- `SHELL` : Shell de connexion configuré.

Le rôle de PATH

Lorsque nous écrivons :

```
python3 programme.py
```

le Shell doit trouver l'exécutable `python3`.

Il recherche notamment dans les répertoires indiqués par la variable `PATH`.

Exemple :

```
echo "$PATH"
command -v python3
```

`PATH` permet de lancer des programmes sans indiquer systématiquement leur chemin complet.

Chapitre 4 — Conditions et tests

1. Le Shell et son fonctionnement
   - Terminal, Shell et commandes
   - Les principaux Shells
   - Les commandes et leurs arguments
2. Les scripts Shell
   - Définition et intérêt
   - Interprétation d'un script
   - Le shebang et les permissions
3. Variables et arguments
   - Les variables
   - Les guillemets
   - Arguments positionnels
   - Variables d'environnement
4. Conditions et tests
   - Principe des conditions
   - Tests numériques et textuels

Prendre des décisions

Un programme doit parfois choisir entre plusieurs actions.

Exemples :

- vérifier qu'un fichier existe ;
- vérifier qu'un argument est fourni ;
- comparer deux valeurs ;
- décider si un programme doit être lancé ;
- afficher un message d'erreur.

Ces décisions sont exprimées avec des **conditions**.

La structure if

Syntaxe générale :

```
if condition; then
commandes
fi
```

Exemple :

```
age=18

if [ "$age" -ge 18 ]; then
echo "Majeur"
fi
```

Le bloc est exécuté uniquement si le test réussit.

if, elif et else

Pour traiter plusieurs situations :

```
note=14

if [ "$note" -ge 16 ]; then
echo "Tres bien"
elif [ "$note" -ge 10 ]; then
echo "Admis"
else
echo "Ajourne"
fi
```

Les conditions sont évaluées dans l'ordre jusqu'à trouver une branche applicable.

Comparer des nombres

Opérateur	Signification
`-eq`	égal
`-ne`	différent
`-lt`	inférieur
`-le`	inférieur ou égal
`-gt`	supérieur
`-ge`	supérieur ou égal

Ces opérateurs sont utilisés pour les comparaisons entières avec la commande de test `[ ]`.

Comparer des chaînes

Pour comparer des chaînes de caractères :

```
nom="Alice"

if [ "$nom" = "Alice" ]; then
echo "Bonjour Alice"
fi
```

Autres exemples :

```
[ "$nom" != "Bob" ]
[ -z "$nom" ]
[ -n "$nom" ]
```

- = : égalité ;

Tester les fichiers et répertoires

Le Shell permet de tester certaines propriétés des fichiers.

```
if [ -f "$1" ]; then
echo "Fichier trouve"
fi
```

Test	Signification
-e	chemin existant
-f	fichier ordinaire
-d	répertoire
-r	lecture autorisée
-w	écriture autorisée
-x	exécution autorisée

Chapitre 5 — Boucles et répétitions

1. Le Shell et son fonctionnement
   - Terminal, Shell et commandes
   - Les principaux Shells
   - Les commandes et leurs arguments
2. Les scripts Shell
   - Définition et intérêt
   - Interprétation d'un script
   - Le shebang et les permissions
3. Variables et arguments
   - Les variables
   - Les guillemets
   - Arguments positionnels
   - Variables d'environnement
4. Conditions et tests
   - Principe des conditions
   - Tests numériques et textuels

Pourquoi utiliser des boucles ?

Une boucle permet de répéter une ou plusieurs instructions.

Elle évite :

- de recopier plusieurs fois le même code ;
- de traiter manuellement chaque fichier ;
- de lancer individuellement plusieurs programmes.

Deux structures principales :

`for` `while`

La boucle for

Une boucle `for` parcourt une liste de valeurs.

```
for i in 1 2 3 4 5
do
echo "$i"
done
```

À chaque itération :

1. une valeur est affectée à `i` ;
2. le corps de la boucle est exécuté ;
3. la valeur suivante est utilisée.

Parcourir des fichiers

Le Shell permet d'utiliser des motifs de fichiers.

```
for fichier in *.txt
do
echo "$fichier"
done
```

Le motif `*.txt` correspond aux fichiers dont le nom se termine par `.txt`.

Les motifs permettent de travailler sur plusieurs fichiers sans connaître leurs noms à l'avance.

La boucle while

La boucle `while` répète des instructions tant qu'une condition réussit.

```
i=1

while [ "$i" -le 5 ]
do
echo "$i"
i=$((i + 1))
done
```

La condition est réévaluée avant chaque itération.

Une boucle dont la condition reste toujours vraie peut devenir infinie.

Arithmétique entière

Bash permet d'effectuer des calculs sur les entiers.

```
a=10
b=3

somme=$((a + b))
produit=$((a * b))
```

Les opérateurs usuels sont :

\+ - * / %

Attention

La division entière ne conserve pas la partie décimale.

Chapitre 6 — Exécution des programmes

1. Le Shell et son fonctionnement
   - Terminal, Shell et commandes
   - Les principaux Shells
   - Les commandes et leurs arguments
2. Les scripts Shell
   - Définition et intérêt
   - Interprétation d'un script
   - Le shebang et les permissions
3. Variables et arguments
   - Les variables
   - Les guillemets
   - Arguments positionnels
   - Variables d'environnement
4. Conditions et tests
   - Principe des conditions
   - Tests numériques et textuels

Lancer un programme depuis Bash

Un script peut exécuter des programmes externes.

```
#!/bin/bash

echo "Debut"

python3 analyse.py donnees.txt

echo "Fin"
```

Bash lance le programme Python, attend normalement sa terminaison, puis poursuit le script.

Le Shell et les processus

Lorsqu'un programme externe est lancé :

1. le Shell demande son exécution ;
2. le système crée ou prépare un processus ;
3. le programme s'exécute ;
4. le Shell récupère son état de terminaison ;
5. l'exécution du script continue.

$$\text{Bash} \longrightarrow \text{Processus Python} \longrightarrow \text{Fin} \longrightarrow \text{Bash}$$

Les codes de retour

Un programme communique un état de terminaison à son environnement.

Convention habituelle :

- 0 : réussite ;
- valeur non nulle : erreur ou autre état particulier.

En Bash, le code de retour de la dernière commande est accessible avec :

```
echo "$?"
```

Les codes de retour permettent aux scripts de détecter certains échecs.

Réagir à une erreur

Exemple :

```
#!/bin/bash

if python3 analyse.py "$1"; then
echo "Analyse reussie"
else
echo "Erreur pendant l'analyse"
exit 1
fi
```

Le Shell utilise directement le code de retour du programme Python.

Bonne pratique

Un script d'automatisation doit vérifier les opérations susceptibles d'échouer.

Chapitre 7 — Bonnes pratiques

1. Le Shell et son fonctionnement
   - Terminal, Shell et commandes
   - Les principaux Shells
   - Les commandes et leurs arguments
2. Les scripts Shell
   - Définition et intérêt
   - Interprétation d'un script
   - Le shebang et les permissions
3. Variables et arguments
   - Les variables
   - Les guillemets
   - Arguments positionnels
   - Variables d'environnement
4. Conditions et tests
   - Principe des conditions
   - Tests numériques et textuels

Écrire des scripts lisibles

Quelques recommandations :

- choisir des noms de variables explicites ;
- indenter les conditions et les boucles ;
- ajouter des commentaires utiles ;
- éviter de recopier inutilement du code ;
- vérifier les arguments ;
- protéger les variables avec des guillemets ;
- afficher des messages d'erreur compréhensibles.

Un script doit pouvoir être relu, compris et modifié facilement.

Commentaires dans un script

Les commentaires permettent de documenter le code.

```
#!/bin/bash

# Recuperer le fichier demande
fichier="$1"

# Verifier son existence
if [ -f "$fichier" ]; then
echo "Fichier trouve"
fi
```

Une ligne commençant par # est généralement ignorée par Bash.

Le shebang constitue un cas particulier.

Exemple d'un script structuré

```
#!/bin/bash

# Verification des arguments
if [ "$#" -ne 1 ]; then
echo "Usage : $0 FICHIER" >&2
exit 1
fi

fichier="$1"

# Verification du fichier
if [ ! -f "$fichier" ]; then
echo "Fichier introuvable" >&2
exit 1
fi

# Execution du traitement
```

Synthèse du cours

Nous avons étudié :

1. le rôle du Shell ;
2. les scripts et leur interprétation ;
3. les variables et arguments ;
4. les conditions et tests ;
5. les boucles ;
6. le lancement de programmes ;
7. les codes de retour ;
8. les bonnes pratiques.

À retenir

Le Shell permet de combiner les fonctionnalités du système et les programmes disponibles pour construire des automatisations.

Vers la suite du cours

Nous savons maintenant :

- lancer des programmes ;
- leur transmettre des arguments ;
- vérifier leurs résultats ;
- répéter des opérations.

Mais comment :

- enregistrer automatiquement leur sortie ?
- transmettre le résultat d'un programme à un autre ?
- traiter des données directement depuis le terminal ?

Prochaine étape : redirections et pipelines